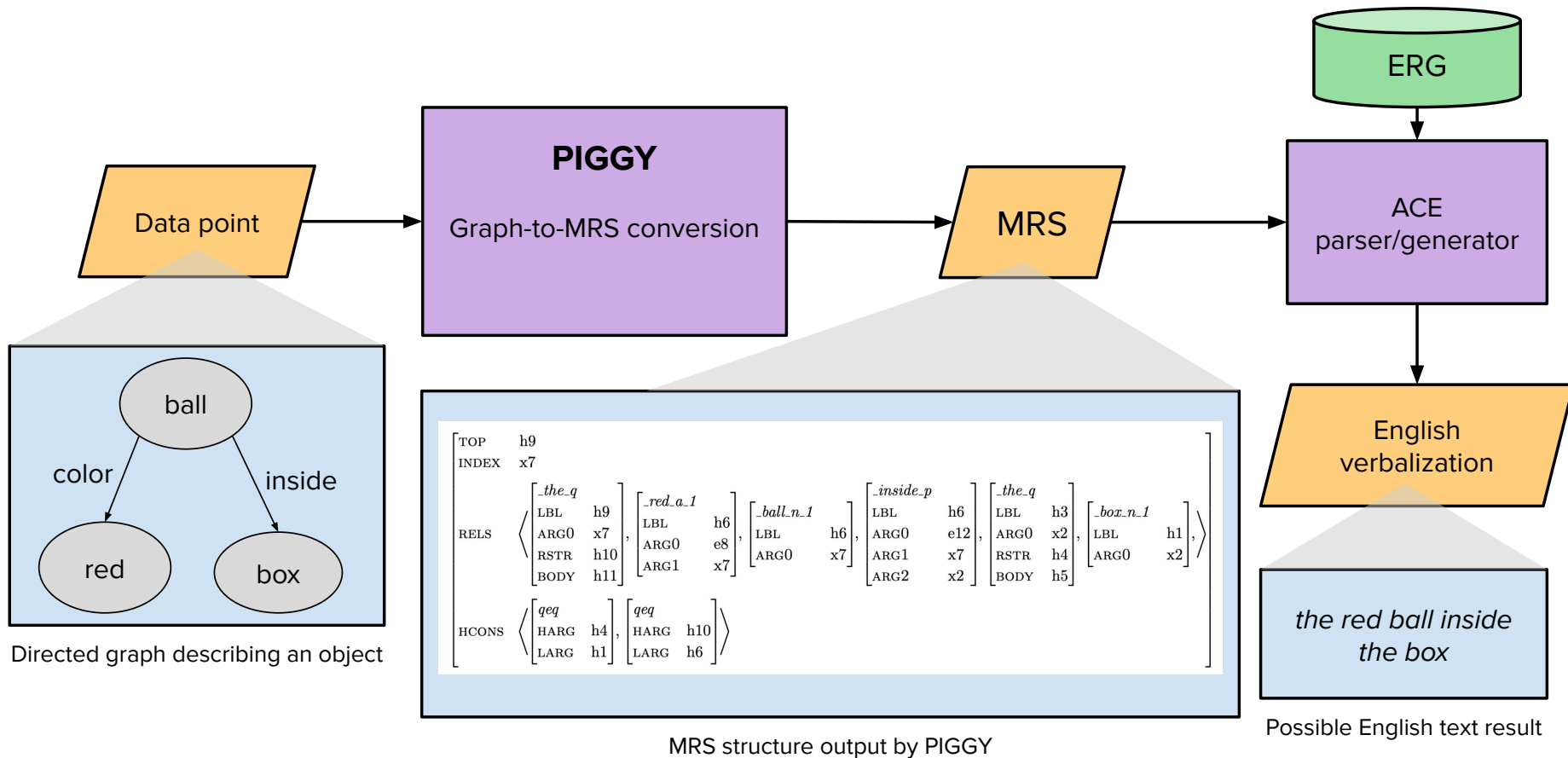


please help me.

things i want feedback about as i finish my dissertation



DEMO

Research Question

- “Are the composition functions used to build semantic representations for data-to-text generation with a computational grammar reusable across different datasets?”
 - Or to put another way, are the functions I create for the SCL reusable in different datasets
- Also, how many functions need to be added when moving to a new dataset?

Topics to discuss

- **Functionality of functions** — how specific/general should the functions be?
- **Naming of functions** — how to handle naming functions transparently while not “implying too much syntax”?
- **Evaluation** — how to compare dissimilar MRSes?

Functionality of Functions

Functionality of functions

- How broad should they be?
- The general goal for the dissertation is “make as few as possible and then leave open the possibility for more specialized ones that are combinations of others”

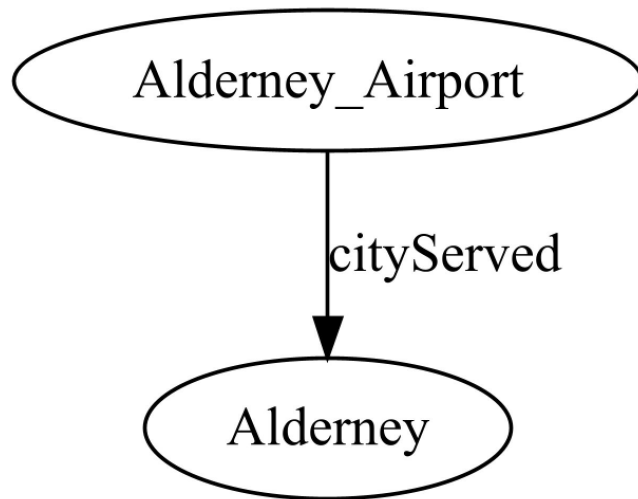
What should the function call for this edge be?

- Possibility 1:

```
transitive_verb_sentence(  
    verb("_serve_v_1"),  
    "parent",  
    "child")
```

- Possibility 2:

```
verb_ARG1(  
    verb_ARG2(  
        verb("_serve_v_1"),  
        "parent",  
        "child")
```



Where I've (maybe) landed

- For now, I think each function should take at most 2 SEMENTs as input
 - so `verb_ARG2 + verb_ARG1` setup
- Then later I can add functions that call more than one of those in sequence
- But part of the motivation for that decision is in service of making my results “look good” for my research question
- Having functions like `verb_ARG1` and `verb_ARG2` would allow me to arrive at the desired result, but is obviously far less convenient than being able to say an edge encodes a transitive sentence

Naming of functions

Naming of functions

- `verb_ARG2` is exactly what i want to avoid
- But `object_of_verb` isn't quite right ...
 - In *active use* the ARG2 of the verb usually(?) is the object, but not in passive use
 - The only thing that remains consistent across both of those MRSeS is the variable that plugs the ARG2 slot
- I do have `object_of_noun` and `object_of_preposition`, but those feel less contentious to me

Single Predicate Functions

- adjective
- comparative_adjective
- conjunction
- determiner
- noun
- number
- preposition
- pronoun
- quantifier
- verb
- name
- non_unique_name
- unknown_adjective
- unknown_noun
- unknown_verb
- manual_synopsis

Compositional Constructions that I'm happy with

- `adjectival_modifier`
- `appositive`
- `copula`
- `date`
- `compound_location`
- `compound_noun`
- `number_with_unit`
- `object_of_noun`
- `object_of_preposition`
- `cardinal_modifier`
- `ordinal_modifier`
- `passive_participle_modifier`
- `possessive`
- `prepositional_modifier`
- `present_participle_modifier`
- `quantify`
- `un_prefix`

Composition Functions I'm not happy with

- `ARG1_relative_clause`
- Name is accurate, but opaque

Composition Functions I'm not happy with

- `ARG1_relative_clause`
- `ARG2_relative_clause`
- Name is accurate, but opaque
- Name is accurate, but opaque

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- Name is accurate, but opaque
- Name is accurate, but opaque
- Different types of coordination...?

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- intransitive_verb_sentence
- Name is accurate, but opaque
- Name is accurate, but opaque
- Different types of coordination...?
- Could be decomposed

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- intransitive_verb_sentence
- negation
- Name is accurate, but opaque
- Name is accurate, but opaque
- Different types of coordination...?
- Could be decomposed
- Different types of negation...?

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- intransitive_verb_sentence
- negation
- relative_direction
- Name is accurate, but opaque
- Name is accurate, but opaque
- Different types of coordination...?
- Could be decomposed
- Different types of negation...?
- Very specific

Composition Functions I'm not happy with

@SemCompTracer.trace Elizabeth Conrad *

```
def relative_direction(self, direction_SEMENT: ..., figure_SEMENT: ..., ground_SEMENT: ...) -> ...:
    """
    1. introduce necessary SEMENTs
        - direction_SEMENT
        - loc_nonsp
        - place_n

    2. ensure ground_SEMENT is quantified
    3. plug ARG2 of direction_SEMENT with quantified_ground, result has functor as hook (appx. "...west of the school")
    4. plug ARG1 of (3) with place_n, result has argument hook (appx. "'place' west of the school")
    5. quantify (4) with `def_implicit_q`
    6. plug ARG2 of loc_nonsp with (5), result has functor hook
    7. plug ARG1 of loc_nonsp with figure_SEMENT, result has argument hook (appx. "house west of the school")
    """
```

Composition Functions I'm not happy with

- ARG1_relative_clause
- ARG2_relative_clause
- coordination
- intransitive_verb_sentence
- negation
- relative_direction
- transitive_verb_sentence
- Name is accurate, but opaque
- Name is accurate, but opaque
- Different types of coordination...?
- Could be decomposed
- Different types of negation...?
- Very specific
- Could be decomposed

Evaluation

Evaluation

- When evaluating my system, I have a number of metrics to evaluate how much of a graph I was able to cover/convert to SEMENTs
- But I also need a way to evaluate the final output
- Right now, I am settling on a few well-known metrics for comparing texts against one another (BLEU, charF)
- But what I am building is MRSEs, and the ERG does the work of generating the text
- What ways are there to evaluate the similarity of MRS structures?